LAPORAN RISET

MODUL 3
“PEMROGRAMAN IMCLAB BERBASIS BLYNK”

oleh :

Evan Adwitiya Dwi Listanto / 23081010093

ROBOTIKA A-801
PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
JAWA TIMUR
2026

1. Tujuan

Percobaan ini bertujuan untuk :

1. Melakukan integrasi modul ESP32 pada Kit iMCLab dengan platform Blynk IoT melalui jaringan WiFi.
2. Mengendalikan motor DC (saklar ON/OFF dan pengaturan kecepatan PWM) melalui *dashboard* Blynk versi *web* maupun *mobile* secara nirkabel, tanpa memerlukan kabel USB ke laptop.
3. Memantau nilai RPM dan PWM motor secara *real-time* langsung dari *dashboard* Blynk.
4. Menganalisis keterkaitan antara konfigurasi *datastream* (Virtual Pin) pada Blynk dengan fungsi yang ada di dalam kode program ESP32.

2. Dasar Teori

1. **Blynk IoT.** Blynk merupakan sebuah platfom IoT yang berbasis cloud yang memungkinkan perangkat seperti ESP32 ke internet dengan identitas berupa *Template ID*, *Template Name*, dan *Auth Token*. Pengguna dapat memantau dan mengendalikan perangkat secara jarak jauh melalui *dashboard* pada aplikasi *mobile* maupun *web* (Blynk.Console) tanpa memerlukan koneksi kabel USB ke laptop. Arsitektur ini membuat perangkat dapat dioperasikan dari mana saja selama terhubung ke jaringan internet.
2. **Datastream dan Virtual Pin.** Datastream adalah sebuah saluran data yang di gunakan untuk menghubungkan widget pada dashboard Blynk dengan kode yang ada pada perangkat ESP32 . alah satu jenisnya adalah *Virtual Pin* (seperti V0, V1, dan seterusnya) yang tidak terikat pada pin fisik GPIO mana pun. Perintah yang dikirim dari *dashboard* akan diterima oleh perangkat melalui fungsi callback `BLYNK_WRITE(Vx)`, sedangkan data dari perangkat dikirimkan ke *dashboard* menggunakan fungsi `Blynk.virtualWrite(Vx, nilai)`. Sifat *hardware-independent* dari *Virtual Pin* ini memudahkan pemindahan kode antar platform perangkat keras yang berbeda.

3. **Pengukuran RPM dengan Sensor Optik.** Pengukuran kecepatan putaran motor dilakukan dengan memanfaatkan piringan berlubang yang dipasang pada poros motor dan melewati celah sensor optik, di mana setiap lubang yang lewat akan menghasilkan satu pulsa. Jumlah pulsa yang terdeteksi dalam selang waktu tertentu kemudian dikonversi menjadi nilai *Revolutions Per Minute* (RPM) menggunakan persamaan $RPM = \frac{P}{H} \times \frac{60}{t}$ dengan P adalah jumlah pulsa, H adalah jumlah lubang pada piringan, dan t adalah waktu pengambilan data dalam detik. Agar tidak ada pulsa yang terlewat, penghitungan pulsa dilakukan menggunakan metode *interrupt* (misalnya *event-driven interrupt*) sehingga mikrokontroler dapat merespons setiap perubahan sinyal sensor secara *real-time*.

3. Alat, Bahan, dan Konfigurasi Pin

Perangkat Keras yang digunakan dalam Percobaan ini :

- ESP 32
- Modul motor IMCLab
- Kabel USB Data
- Kabel Power 5V 2 A
- Laptop
- Smartphone

Perangkat Lunak yang digunakan dalam Percobaan ini :

- Arduino IDE Versi 2.3.10
- Aplikasi Blynk IoT di HP
- Blynk Console di browser
- Versi core ESP32 di Boards Manager : 2.0.17
- Jaringan Wifi

Tabel 1. Pin pada ESP32

Fungsi di kode	Variabel di kode	GPIO di kode	Label di papan
LED indikator	ledPin	2	D2
Pin kontrol arah motor 1	motor1Pin1	27	D27
Pin kontrol arah motor 2	motor1Pin2	26	D26
Enable (PWM 30 kHz, 8 bit)	enable1Pin	12	D12
Sensor RPM (interrupt FALLING)	pin_rpm	13	D13

Tabel 2. Datastream pada template

Nama	Pin	Tipe data	Min	Maks	Satuan	Arah dan fungsi
RPM	V0	Double	0	13000	rpm	Perangkat ke dashboard: kecepatan putar
PWM	V1	Integer	0	255	BIT	Perangkat ke dashboard: nilai PWM aktual
PMW MOTOR	V2	Integer	0	255	BIT	Dashboard ke perangkat: slider PWM
Power Switch	V3	Integer	0	1	-	Dashboard ke perangkat: saklar ON/OFF

4. Prosedur Percobaan

1. Buka website Blyn.Console melalui browser dan masuk menggunakan akun Blynk yang sudah terdaftar.

2. Pilih Menu Developer zone > My Template > New Template
3. Kemudian isi nama template pilih Hardware ESP32, dan tentukan tipe koneksi WiFi, lalu klik Done.
4. Pada halaman template yang baru kita buat, buka tab Datasream > New Datasream Pada halaman template yang baru dibuat, buka tab Datastreams dan klik tombol "+ New Datastream", kemudian pilih Virtual Pin.
5. Buat Datastream pertama (V0) dengan nama "RPM", ", atur Data Type menjadi Integer, nilai Min 0 dan Max 3000.
6. Buat Datastream kedua (V1) dengan nama "Kecepatan PWM", atur Data Type menjadi Integer, nilai Min 0 dan Max 255.
7. Buat Datastream ketiga (V2) dengan nama "PWM motor", atur Data Type menjadi Integer, nilai Min 0 dan Max 255.
8. Buat Datastream pertama (V3) dengan nama "Power Switch", atur Data Type menjadi Integer, nilai Min 0 dan Max 1.
9. Klik Save pada masing-masing datastream untuk memastikan Virtual Pin (V0 hingga V3) tersimpan dengan benar
10. Pada halaman template di Blynk.Console, buka tab Web Dashboard (atau Mobile Dashboard jika menggunakan aplikasi HP). Klik ikon Edit (berbentuk pensil) di pojok kanan atas untuk mulai mengatur tampilan.
11. Klik tombol + (Add Widget) atau kotak kosong pada area dashboard untuk memunculkan menu pilihan widget, lalu pilih Gauge.
12. Klik widget Gauge yang baru ditambahkan, lalu pada menu pengaturan di sebelah kanan, atur Datastream ke V0 (RPM). Ubah judul widget menjadi "RPM" dan atur mode tampilan menjadi Gauge
13. Tambahkan widget baru dengan klik tombol + lagi, lalu pilih Radial Gauge. Atur Datastream ke V1 (PWM), beri judul " PWM", dan atur nilai Min = 0 serta Max = 255.
14. Tambahkan widget ketiga dengan klik +, lalu pilih Slder lalu pilih Slider. Atur Datastream ke V2 (PWM Motor), beri judul " PWM", dan atur nilai Min = 0 serta Max = 255.

15. Tambahkan widget keempat dengan klik +, lalu pilih Switch. Atur Datastream ke V3 (Power Switch), beri judul " Power Switch ", dan atur mode tampilan menjadi Switch (ON/OFF).
16. Setelah keempat widget selesai ditambahkan, atur tata letak (posisi dan ukuran) masing-masing widget dengan cara drag and drop agar tampilan dashboard rapi dan mudah digunakan.
17. Klik tombol Save (ikon disket) di pojok kanan atas untuk menyimpan perubahan. Pastikan tampilan dashboard sudah sesuai sebelum diuji coba pada perangkat keras.
18. Hubungan Modul ESP 32 Ke Laptop menggunakan kabel USB data.
19. Pastikan Kabel USB yang digunakan mendukung Transfer data, bukan hanya untuk pengisian daya.
20. Periksa Apakah ESP32 sudah terdeteksi pada komputer.
21. Buka aplikasi Arduino IDE.
22. Salin dan tempel Kode berikut ke Arduino IDE.

    Kode `imclab_Blynk_kelas.ino`

```
/*************************************************************
 * iMCLab + ESP32 + Blynk IoT
 * RPM Monitoring dan Kontrol PWM
 *************************************************************/

#define BLYNK_TEMPLATE_ID "ID Template Blynk"
#define BLYNK_TEMPLATE_NAME "Nama Template Blynk"
#define BLYNK_AUTH_TOKEN "Token Template Blynk"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

//Konfigurasi Wifi
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "SSID wifi";
char pass[] = "Password";


// ============================================================
// Motor dan LED
// ============================================================

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// ============================================================
// Sensor RPM
```

```
// ============================================================

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;

// ============================================================
// PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

// ============================================================
// RPM
// ============================================================

float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================

String inputString = "";

BlynkTimer timer;

// ============================================================
// ISR Encoder
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}

// ============================================================
// Sync setelah reconnect Blynk
// ============================================================

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);
}
```

```
// ============================================================
// Hitung RPM
// ============================================================

void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

// ============================================================
// Terapkan PWM ke Motor
// ============================================================

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Status Serial
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

// ============================================================
// Kirim Data ke Blynk
// ============================================================
```

```
void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Slider PWM dari Blynk (V2)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}

// ============================================================
// Tombol ON/OFF dari Blynk (V3)
// ============================================================

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = 200;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

// ============================================================
// Setup
// ============================================================

void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
```

```
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

//  Blynk.begin(
//    BLYNK_AUTH_TOKEN,
//    ssid,
//    pass);

  // Koneksi ke Blynk
  Blynk.begin(auth, ssid, pass);

  timer.setInterval(
    1000L,
    kirimKeBlynk);

  Serial.println("================================");
  Serial.println("iMCLab + Blynk Ready");
  Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
}

// ============================================================
// Loop
// ============================================================

void loop()
{
  Blynk.run();
  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();
```

```
  if (c == '\n' || c == '\r')
  {
    inputString.trim();

    if (inputString.startsWith("PWM:"))
    {
      int val = inputString.substring(4).toInt();

      val = constrain(
        val,
        DUTY_MIN,
        DUTY_MAX);

      dutyCycle = val;
      motorEnable = (dutyCycle > 0);

      applyMotor();

      tampilkanStatus("Set PWM");
    }

    else if (inputString.length() > 0)
    {
      char cmd = inputString.charAt(0);

      if (cmd == '1')
      {
        if (dutyCycle == 0)
          dutyCycle = 200;

        motorEnable = true;

        applyMotor();

        tampilkanStatus("Motor ON");
      }

      else if (cmd == '0')
      {
        dutyCycle = 0;
        motorEnable = false;

        applyMotor();

        Serial.println("Motor OFF");
      }

      else if (cmd == '+')
      {
        dutyCycle += DUTY_STEP;

        if (dutyCycle > DUTY_MAX)
          dutyCycle = DUTY_MAX;

        motorEnable = true;

        applyMotor();

        tampilkanStatus("PWM NAIK");
      }

      else if (cmd == '-')
```

```
        {
          dutyCycle -= DUTY_STEP;

          if (dutyCycle < DUTY_MIN)
            dutyCycle = DUTY_MIN;

          motorEnable = (dutyCycle > 0);

          applyMotor();

          tampilkanStatus("PWM TURUN");
        }

        else if (cmd == 'R' || cmd == 'r')
        {
          Serial.print("RPM = ");
          Serial.print(rpm, 1);
          Serial.print(" | PWM = ");
          Serial.println(dutyCycle);
        }
      }

      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

23. Ganti Bagian salin Template ID, Template Name, dan Auth Token Sesuai dengan yang telah anda buat di Blynk

```
#define BLYNK_TEMPLATE_ID "ID Template Blynk"
#define BLYNK_TEMPLATE_NAME "Nama Template Blynk"
#define BLYNK_AUTH_TOKEN "Token Template Blynk"
```

24. Sesuaikan ssid[] dan pass[] dengan nama dan password WiFi 2,4 GHz. Pastikan penulisan case-sensitive dan tanpa spasi tambahan.

```
//Konfigurasi Wifi
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "SSID wifi";
char pass[] = "Password";
```

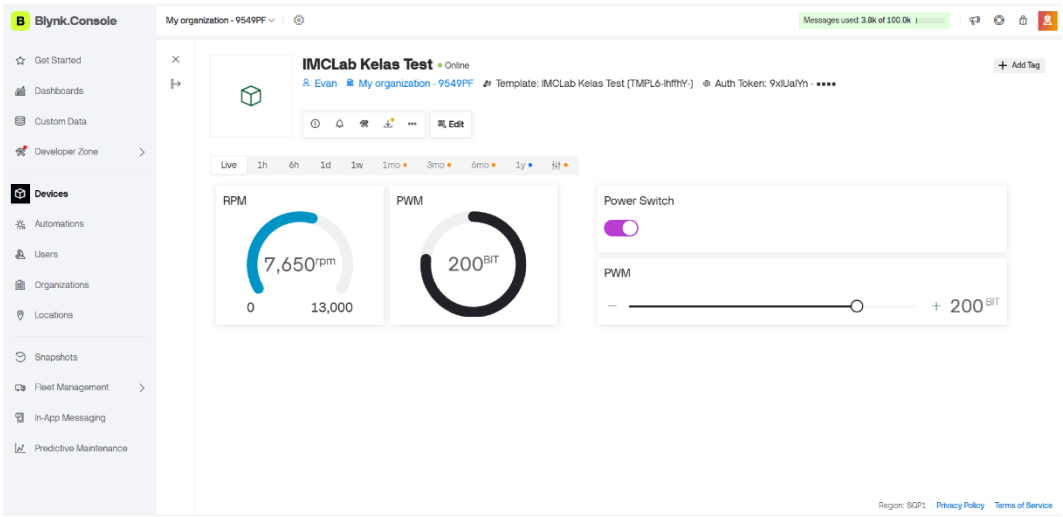
25. Pilih board ESP32 melalui menu Tools > Board > esp32 > DOIT ESP32 DEVKIT V1 .
26. Pilih port melalu menu Tools > Port >(Pilih port yang sesuai).
27. Klik Tombol Upload
28. Tunggu hingga proses upload selesai.
29. Buka browser dan akses Blynk.Console, atau buka aplikasi Blynk di HP.
30. Masuk ke menu Devices, lalu periksa status perangkat. Pastikan indikator perangkat berwarna hijau dengan keterangan Online.
31. Pada tampilan dashboard Blynk, cari widget Switch (Saklar Power).

32. Tekan Switch tersebut ke posisi ON.
33. Amati motor DC pada kit iMCLab, pastikan motor mulai berputar.
34. Geser widget Slider (PWM) ke kanan atau ke nilai tertentu (misalnya ke angka 150).
35. Amati perubahan pada motor; putaran motor akan menjadi lebih cepat atau lebih lambat sesuai posisi slider.
36. Perhatikan widget Gauge atau Label (RPM dan PWM) di dashboard Blynk.
37. Pastikan angka yang tertera pada layar berubah secara real-time mengikuti kecepatan putaran motor.
38. Tekan kembali Switch ke posisi OFF, lalu pastikan motor berhenti berputar dan nilai RPM pada dashboard kembali menjadi 0.

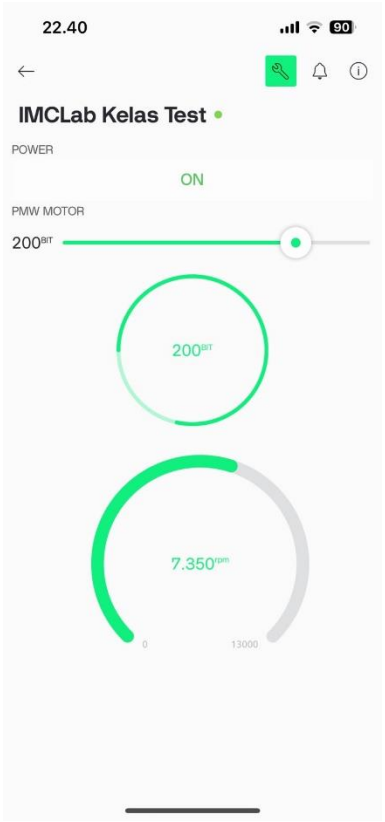
5. Hasil Percobaan

1. Dashboard saat motor berputar (ON)

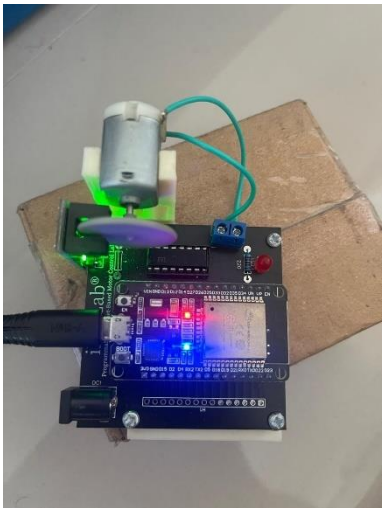
   Gambar 5.1 (web) dan Gambar 5.2 (mobile) menunjukkan perangkat berstatus Online dengan saklar Power ON dan nilai PWM serta RPM yang tampil di dashboard.



Gambar 6.1. Dashboard web (Blynk.Console): perangkat Online, saklar ON, PWM 200, RPM 7.650



Gambar 6.2. Dashboard mobile: perangkat Online, saklar ON, PWM 200, pengukur RPM menampilkan 7.350



Gambar 6.3. Motor ESP32 ON dengan PWM 200

Tabel 6.1. Nilai yang terbaca pada dua dashboard. Keduanya diambil pada waktu berbeda (jam di layar HP 22.40 dan nama file tangkapan web 22:41), sehingga nilainya tidak harus sama.

Dashboard	Status	Saklar Power	PWM (BIT)	RPM
Mobile (Gambar 6.2)	Online	ON	200	7.350
Web (Gambar 6.1)	Online	ON	200	7.650

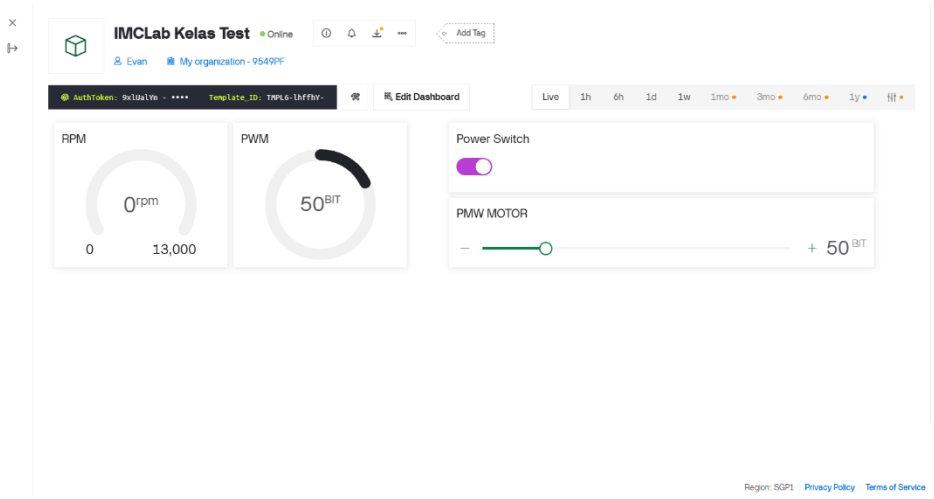
2. Perhitungan RPM dari nilai di dashboard

   Nilai 7.650 rpm pada Gambar 6.1 dapat diuraikan dengan rumus di kode (RPM = pulsa / 2 x 60 untuk jendela 1 detik). Jumlah pulsa tidak terbaca langsung dari dashboard, sehingga angkanya diturunkan dari nilai RPM:

Besaran	Nilai	Cara memperoleh
RPM di dashboard	7.650	Dibaca dari Gambar 6.1
Putaran per detik	127,5	7.650 / 60
Pulsa per detik (rev)	255	127,5 x 2 lubang
RPM = (255 / 2) x 60	7.650	Cocok dengan nilai di dashboard

3. Hubungan PWM dan RPM

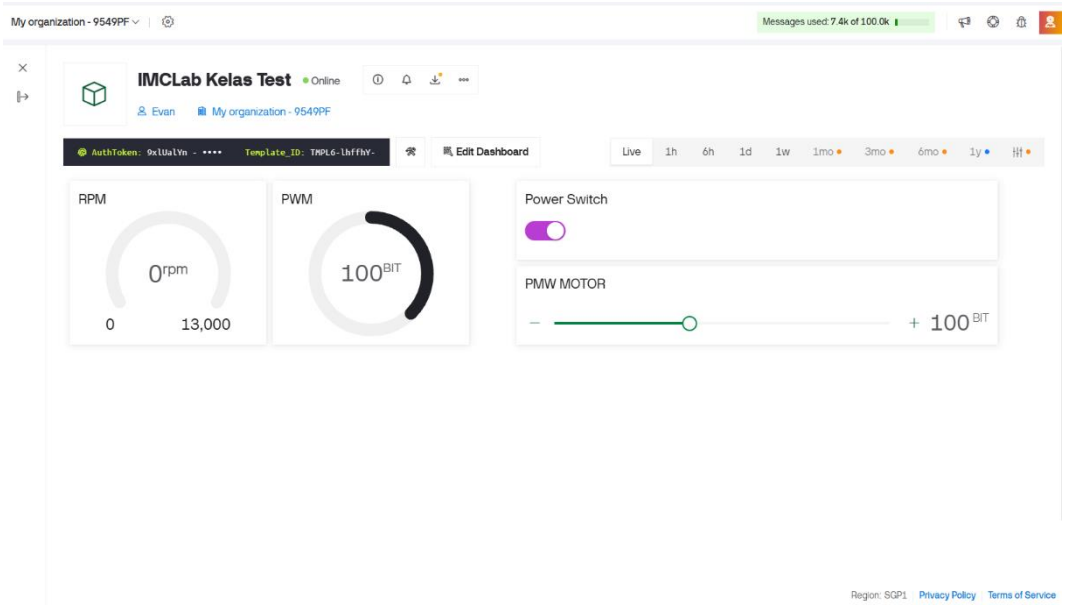
PWM	RPM percobaan 1	RPM percobaan 2	RPM percobaan 3	Rata-rata
50	0	0	0	0
100	0	0	0	0
150	2.100	2.340	2.280	2.240
200	7.350	7.650	8.340	7.780



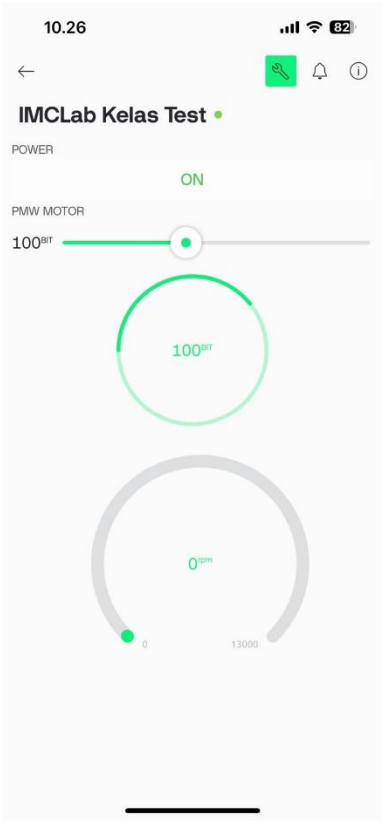
Gambar 6.3. Dashboard web (Blynk.Console): perangkat Online, saklar ON, PWM 50, pengukur RPM menampilkan 0



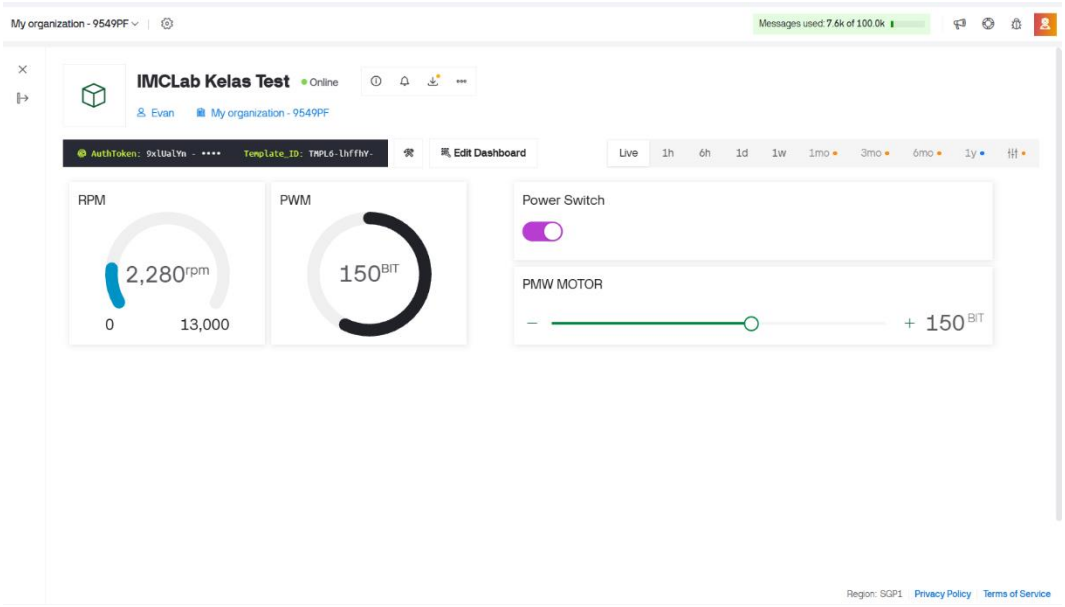
Gambar 6.4. Dashboard mobile: perangkat Online, saklar ON, PWM 50, pengukur RPM menampilkan 0



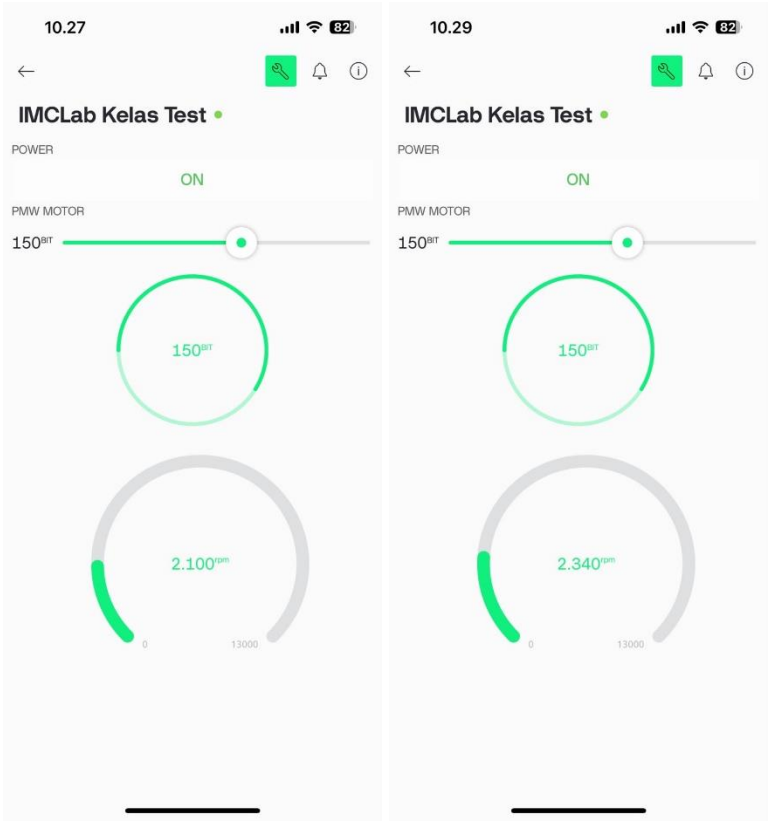
Gambar 6.5. Dashboard web (Blynk.Console): perangkat Online, saklar ON, PWM 100, pengukur RPM menampilkan 0



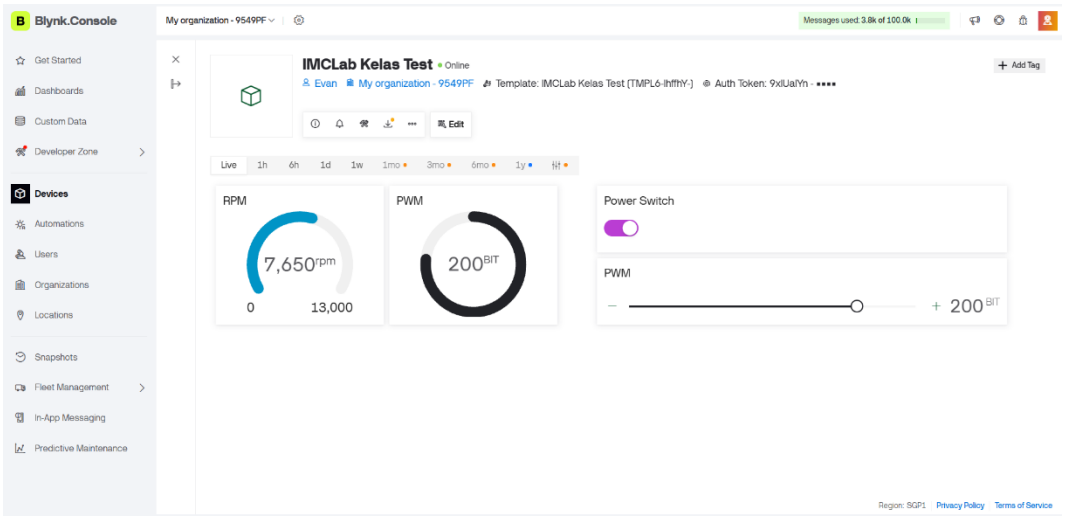
Gambar 6.6. Dashboard mobile: perangkat Online, saklar ON, PWM 100, pengukur RPM menampilkan 0



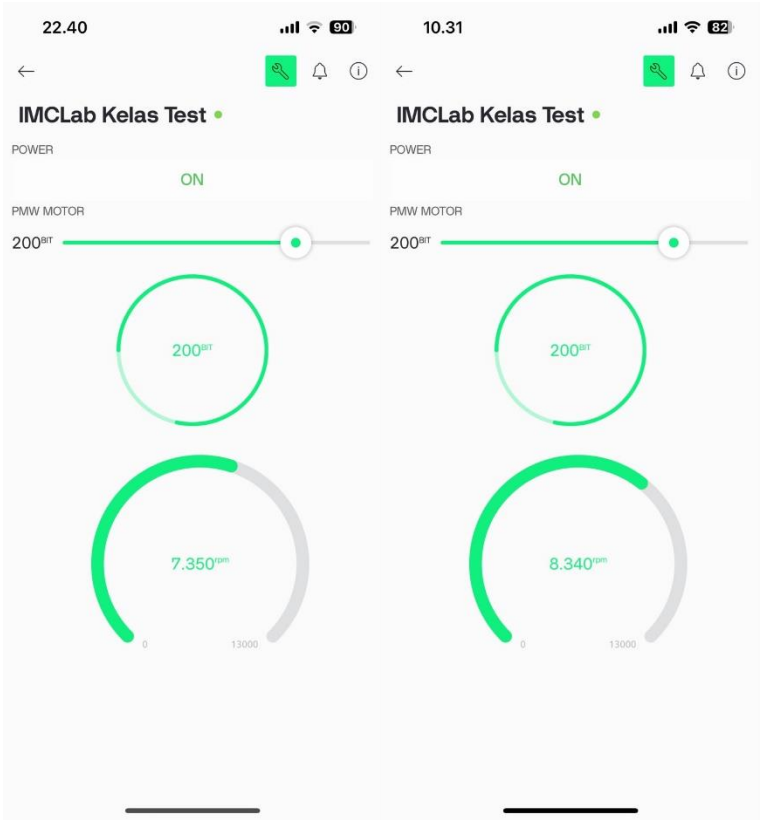
Gambar 6.7. Dashboard web (Blynk.Console): perangkat Online, saklar ON, PWM 150, pengukur RPM menampilkan 2280



Gambar 6.8. Dashboard mobile: perangkat Online, saklar ON, PWM 150, pengukur RPM menampilkan 2100 & 2340



Gambar 6.9. Dashboard web (Blynk.Console): perangkat Online, saklar ON, PWM 200, pengukur RPM menampilkan 7650



Gambar 6.10. Dashboard mobile: perangkat Online, saklar ON, PWM 200, pengukur RPM menampilkan 7350 & 8340

6. Analisis

1. Sistem ini menerapkan mekanisme komunikasi data dua arah. Perintah kontrol dikirim dari dashboard ke ESP32 melalui Virtual Pin V2 dan V3 menggunakan fungsi `BLYNK_WRITE`. Sebaliknya, data dari ESP32 dikirim ke dashboard melalui V0 dan V1 setiap 1 detik menggunakan `BlynkTimer`. Selain itu, fungsi `applyMotor()` juga akan memperbarui nilai pada V1, V2, dan V3 setiap kali nilai duty cycle berubah. Mekanisme ini memastikan bahwa posisi slider dan saklar pada dashboard akan ikut menyesuaikan secara otomatis, bahkan jika perintah kontrol awal dikirim melalui Serial Monitor. Hasilnya, dashboard akan selalu merepresentasikan kondisi motor yang sebenarnya secara akurat.
2. Resolusi pengukuran nilai RPM dipengaruhi oleh metode penghitungan pulsa dalam interval waktu 1 detik dan penggunaan piringan dua lubang. Dalam kondisi ini, setiap satu pulsa merepresentasikan 0,5 putaran atau setara dengan 30 RPM. Konsekuensinya, nilai RPM yang ditampilkan pada sistem akan selalu merupakan kelipatan 30. Sebagai contoh, nilai 8.160 RPM diperoleh dari 272 pulsa dikalikan dengan 30, yang konsisten dengan karakteristik tersebut. Pada kecepatan putaran rendah, tingkat resolusi ini tergolong kasar (misalnya, selisih 30 RPM pada nilai 300 RPM mencapai 10%), namun pada putaran tinggi, pengaruhnya menjadi relatif kecil.
3. Terdapat beberapa keterbatasan dan catatan keamanan pada sistem ini. Fungsi Blynk.begin(auth, ssid, pass) umumnya bersifat blocking, yaitu menahan eksekusi program hingga koneksi WiFi dan server Blynk berhasil terbentuk. Akibatnya, selama perangkat belum berstatus online, kontrol melalui Serial Monitor tidak dapat dijalankan. Dari sisi keamanan, Auth Token dan password WiFi ditanam langsung (hardcode) di dalam kode, yang berisiko bocor atau disalahgunakan jika kode tersebut diunggah ke repositori publik seperti GitHub.

7. Kesimpulan

1. Modul ESP32 pada Kit iMCLab berhasil terhubung ke platform Blynk dan berstatus Online pada antarmuka web maupun mobile (Gambar 6.1 dan 6.2).
2. Motor DC dapat dinyalakan, dimatikan, dan diatur kecepatannya melalui dashboard Blynk (Virtual Pin V3 dan V2) secara nirkabel tanpa memerlukan

kabel USB ke laptop. Hal ini dikonfirmasi melalui pengamatan fisik putaran motor dan dokumentasi video.

3. Nilai RPM dan PWM berhasil ditampilkan secara real-time pada dashboard . Nilai RPM tersebut dihitung berdasarkan pembacaan pulsa dari sensor optik yang dipasang pada piringan berlubang (2 lubang).
4. Konfigurasi Datastream (Virtual Pin) terbukti berfungsi sebagai jembatan komunikasi antara widget pada dashboard dengan fungsi BLYNK_WRITE (untuk menerima perintah) dan virtualWrite (untuk mengirim data) di dalam kode ESP32.